

UNIVERSITE DE L'ENTREPRISE



Créativité – Professionnalisme – Bonne gouvernance

DIRECTION DE L'ENSEIGNEMENT SUPERIEUR PRIVE

Master 1 SRT

PROJET DE DBA NOSQL

Système Temps Réel de Recommandation et Traçabilité pour une Plateforme de E-Commerce (Scale-Out Architecture)

Rédigé par :

- Ange Fabrice Otsouandjogo Opigas
- MIDOUMBI NZOGHE Alex Aimé
- BIROUNGOU T Lilian

2025-2026

SOMMAIRE

INTRODUCTION.....	1
2. Architecture polyglotte : choix techniques.....	1
2.1 Le compromis CAP	1
3. Étape 1 : Qualité des données (nettoyage)	2
3.1 Règles de validation appliquées	2
3.2 Résultats obtenus	2
4. Étape 2 : Infrastructure Docker	3
4.1 Le Replica Set MongoDB	3
4.2 Persistance Redis	3
5. Étape 3 : Injection en masse	3
5.1 Modélisation MongoDB.....	3
5.2 Modélisation Neo4j	3
5.3 Performance de l'injection (opérations par lots).....	4
6. Étape 4 : API	4
6.1 Agrégations MongoDB	4
6.2 Recommandations Neo4j	4
<i>Exemple de requête Cypher (profondeur 2)</i>	5
6.3 Rate limiting via Redis	5
7. Test de résilience et théorème CAP	5
8. Limites identifiées et pistes d'amélioration	6
8.1 Biais de popularité dans les recommandations.....	6
8.2 Autres limites	6
CONCLUSION	7

INTRODUCTION

Ce projet consiste à concevoir le back-end d'une plateforme e-commerce à forte charge, en s'appuyant sur trois technologies NoSQL complémentaires plutôt qu'une base relationnelle unique. Ce choix répond à un principe central des architectures modernes à grande échelle : utiliser la base la mieux adaptée à chaque type de requête, plutôt que de forcer un seul modèle de données à répondre à des besoins très différents (persistance transactionnelle, navigation relationnelle, accès en mémoire à très faible latence).

Le projet couvre l'intégralité du cycle de vie de la donnée : nettoyage d'un jeu de données brut de plus de 100 000 transactions, modélisation et injection en masse dans les trois bases, puis exposition de ces données via une API REST démontrant des cas d'usage réels (agrégations, recommandations, limitation de débit).

2. Architecture polyglotte : choix techniques

Technologie	Modèle	Rôle dans le projet	Pourquoi ce choix
MongoDB	Document	Catalogue produits, commandes	Pas de JOIN : une commande et ses articles tiennent dans un seul document, lu en une requête
Neo4j	Graphe	Moteur de recommandation	Navigation naturelle entre entités liées (Client → Produit → Client) impossible à exprimer efficacement en SQL/document
Redis	Clé-valeur (mémoire)	Sessions à expiration, top ventes temps réel	Latence quasi nulle, structures natives adaptées (Hash + TTL, Sorted Set) pour du temps réel

2.1 Le compromis CAP

Le théorème CAP énonce qu'un système distribué ne peut garantir simultanément que deux des trois propriétés suivantes : Cohérence (Consistency), Disponibilité (Availability) et Tolérance au Partitionnement (Partition tolerance). Chacune des trois bases fait un compromis différent :

- MongoDB (avec write concern « majority ») privilégie CP : en cas de perte du nœud primaire, le cluster refuse temporairement les écritures le temps qu'un nouveau primaire soit élu, plutôt que de risquer une incohérence.

- Redis, en configuration simple (sans Sentinel/Cluster), privilégie la disponibilité et la performance au prix d'une cohérence plus faible en cas de panne.
- Neo4j, en instance unique dans ce projet, n'est pas distribué : la problématique CAP ne s'applique pas directement, mais le choix a été fait consciemment (une architecture Causal Cluster existe pour la haute disponibilité en production).

3. Étape 1 : Qualité des données (nettoyage)

Le fichier source (ecommerce_raw_transactions_dirty.csv, 101 500 lignes) contient des anomalies représentatives d'un vrai système logistique legacy. Le nettoyage est entièrement vectorisé avec pandas (aucune boucle ligne par ligne), et chaque rejet est journalisé avec son motif dans errors.log.

3.1 Règles de validation appliquées

Anomalie	Règle de détection	Traitement
Doublons stricts	Ligne identique à une ligne déjà vue	Supprimée
customer_id manquant	Champ vide/absent	Conservée (flag is_anonymous), exclue de Neo4j
Quantité incohérente	quantity <= 0 ou non numérique	Rejetée
Prix corrompu	Contient "CFA", négatif ou nul	Rejetée
Date invalide	Hors format ISO 8601 ou date inexistante	Rejetée

3.2 Résultats obtenus

Indicateur	Valeur
Lignes brutes	101 500
Doublons stricts supprimés	1 500
Quantités incohérentes rejetées	2 027
Prix corrompus rejetés	1 009
Dates invalides rejetées	1 982
Total rejeté	6 518 (6,4 %)
Lignes propres conservées	94 982

Indicateur	Valeur
dont transactions anonymes	2 020
dont éligibles Neo4j	92 962

4. Étape 2 : Infrastructure Docker

L'infrastructure est entièrement définie en Docker Compose et comprend un Replica Set MongoDB à 3 nœuds, une instance Neo4j (avec le plugin APOC) et une instance Redis configurée avec persistance AOF + RDB.

4.1 Le Replica Set MongoDB

Les 3 nœuds (mongo1, mongo2, mongo3) démarrent symétriques, puis un conteneur « one-shot » (mongo-setup) exécute rs.initiate() pour constituer le cluster, avec mongo1 en priorité haute (candidat primaire naturel). Le script attend activement l'élection d'un primaire avant de poursuivre, plutôt qu'un simple délai fixe — ce point s'est révélé nécessaire en pratique, l'élection pouvant prendre de quelques secondes à plusieurs dizaines de secondes selon la machine.

4.2 Persistance Redis

Redis est configuré avec le mode AOF (Append Only File, journal de chaque écriture, fsync chaque seconde) combiné à des snapshots RDB périodiques. Ce double mécanisme garantit qu'un redémarrage du conteneur ne fait perdre ni les sessions actives ni le classement des ventes.

5. Étape 3 : Injection en masse

5.1 Modélisation MongoDB

Chaque transaction nettoyée devient un document unique dans la collection orders, avec un tableau items imbriqué. Ce choix évite tout JOIN : une commande complète se lit en une seule requête. Le CSV source étant structuré 1 ligne = 1 article, chaque document contient ici un seul élément dans items mais la structure reste extensible à de vrais paniers multi-articles.

5.2 Modélisation Neo4j

Le graphe relie les nœuds Customer et Product par une relation PURCHASED, porteuse de la date et de la quantité. Les transactions anonymes sont exclues (pas de nœud Customer possible).

Des contraintes d'unicité sur `customer_id` et `product_id` garantissent des opérations MERGE efficaces (indexation automatique sous-jacente).

5.3 Performance de l'injection (opérations par lots)

Aucune insertion n'est faite ligne par ligne. MongoDB utilise `bulk_write()` par lots de 5000 documents ; Neo4j utilise UNWIND pour traiter des lots de 2000 lignes en une seule requête Cypher ; Redis utilise un pipeline pour grouper les commandes réseau.

Base	Volume traité	Temps mesuré
MongoDB	94 982 documents	≈ 6,6 s
Neo4j	92 962 relations PURCHASED	≈ 34,3 s
Redis	94 982 incréments (Sorted Set) + 200 sessions	≈ 2,7 s

6. Étape 4 : API

L'API est développée avec FastAPI et expose trois familles d'endpoints.

6.1 Agrégations MongoDB

Endpoint	Description
GET /aggregations/revenue-by-category	CA total et nombre de commandes par catégorie de produit
GET /aggregations/top-customers	Classement des clients par montant total dépensé
GET /aggregations/sales-over-time	Évolution du CA par mois ou par jour

6.2 Recommandations Neo4j

Deux stratégies complémentaires sont implémentées, illustrant deux approches classiques de recommandation :

- **Clients similaires (collaborative filtering)**

GET /recommendations/customer/{customer_id}?depth=2|3: on part d'un client, on remonte vers les produits qu'il a achetés, on redescend vers d'autres clients ayant les mêmes achats, puis on ressort leurs autres produits. Le paramètre `depth` contrôle si l'on s'arrête après un seul saut « produit ↔ client » (profondeur 2) ou si l'on va chercher une couche supplémentaire, plus proche des « amis des amis » (profondeur 3).

- **Produits achetés ensemble** GET /recommendations/product/{product_id} : à partir d'un produit, on cherche les autres produits achetés par les mêmes clients le même jour. Le CSV source étant structuré 1 ligne = 1 article (sans identifiant de panier explicite), la co-occurrence « même client, même jour » sert de proxy raisonnable au panier.

Exemple de requête Cypher (profondeur 2)

```
MATCH (c:Customer {customer_id: $customer_id})
      -[:PURCHASED]->(:Product)<-[:PURCHASED]-(o:Customer)
      -[:PURCHASED]->(rec:Product)
WHERE NOT (c)-[:PURCHASED]->(rec)
RETURN rec.product_id, rec.category, count(DISTINCT o) AS score
ORDER BY score DESC LIMIT $limit
```

6.3 Rate limiting via Redis

Chaque endpoint (hors /health) est protégé par une dépendance FastAPI qui applique un algorithme de fenêtre fixe (fixed window) : une clé Redis ratelimit:{ip}:{minute} est incrémentée (INCR) à chaque requête et expire automatiquement (EXPIRE) à la fin de la minute. Au-delà de la limite configurée (60 requêtes/minute par défaut), l'API renvoie un code HTTP 429.

Limite connue de cette approche, à mentionner à l'oral : un client peut envoyer jusqu'à deux fois la limite en rafale à la frontière entre deux fenêtres (ex. juste avant et juste après le changement de minute). Un algorithme de fenêtre glissante (sliding window log) serait plus précis mais plus coûteux en mémoire Redis un compromis délibéré pour ce projet.

7. Test de résilience et théorème CAP

Le Replica Set MongoDB a été testé en coupant le nœud primaire (docker stop mongo1) pendant une insertion continue. Comportement observé et attendu :

- Pendant l'élection (quelques secondes), le cluster refuse les écritures : il n'y a temporairement plus de primaire c'est le compromis CP illustré concrètement.
- Un nouveau primaire (mongo2 ou mongo3) est élu automatiquement, sans intervention manuelle.
- Les écritures reprennent dès l'élection terminée, sans perte de données déjà validées avec write concern majority.

Lorsque l'ancien nœud revient (docker compose up -d mongo1), il rejoint le cluster comme secondaire et se resynchronise automatiquement.

Ce test correspond exactement à la démonstration annoncée par l'enseignant en soutenance (coupure d'un nœud en pleine transaction).

8. Limites identifiées et pistes d'amélioration

8.1 Biais de popularité dans les recommandations

Les tests ont révélé qu'un algorithme de filtrage collaboratif naïf (profondeur 2 ou 3) tend à sur-représenter les produits déjà très populaires : ceux-ci sont mécaniquement achetés par un grand nombre de « clients similaires », ce qui gonfle leur score indépendamment des goûts réels du client ciblé. Exemple observé : pour un client donné, les deux premières recommandations obtiennent un score de 1572 et 1565, contre 373 à 403 pour les suivantes — ces deux produits figurent aussi dans le top des ventes globales.

Piste d'amélioration : pondérer le score par l'inverse de la popularité globale du produit (logique proche d'un TF-IDF), ou combiner le score de similarité avec une affinité de catégorie, pour limiter la domination des produits déjà massivement vendus.

8.2 Autres limites

- Neo4j tourne en instance unique dans ce projet (pas de cluster de haute disponibilité), contrairement à MongoDB.
- Le rate limiting en fenêtre fixe autorise une rafale à la frontière de deux fenêtres (cf. section 6.3).
- La détection de « panier » pour les recommandations Neo4j est un proxy (même client, même jour) faute d'identifiant de panier explicite dans les données sources.

CONCLUSION

Ce projet a permis de mettre en œuvre une architecture polyglotte complète et fonctionnelle, du nettoyage de données brutes jusqu'à une API exposant des cas d'usage réels sur les trois bases. Chaque choix technique modélisation documentaire imbriquée, graphe de recommandation, structures en mémoire avec TTL a été justifié par les besoins spécifiques de chaque type de requête plutôt que par une contrainte imposée. Les limites identifiées (biais de popularité, fenêtre de rate limiting) ne remettent pas en cause l'architecture, mais illustrent une compréhension critique de ses compromis un point valorisé à l'oral autant que le fonctionnement du code lui-même.